CIS 415 Operating Systems

Project 3 Report Collection

Submitted to: Prof. Allen Malony

Author: Kai Hogan

Duck ID: khogan

95 Number: 951967129

Introduction:
This project implements a roller coaster simulation in C using different threads. Each passenger is represented by a thread and each car is represented by a thread. Passengers wait in the park then get in line for a ticket and then get in line for a ride once they get a ticket. Passengers then wait for a car and board once a car has space to load them on. Once a car has loaded enough passengers on or a certain amount of time has passed the car exits and rides around the track. It's important that cars load in order and passengers enqueue for the ride and enter the rides in order.

Background:
I used the Pthread_Create function to create the pthreads and initialize their routine. I used the Pthread cancel function to stop the threads and the pthread_join function to free all of the memory for the threads. I used pthread_mutex_lock() and pthread_mutex_unlock() to make sure no race conditions occurred. I also used a semaphore to limit how many passengers could join the ride queue. In addition, I used the time.h library and its associated functions to keep track of the time and print it to the console. I also used some structs I created myself in order to implement some of the project functionalities.

Implementation: Part three had seven different C files (including Park.c). I delegated a lot of different functionalities to different files and structs I created. I created a queue structure to implement a ride queue and a ticket queue for passengers. I created a Car structure that kept a queue of passengers as well as the car's status and remaining capacity. I created a simulation stats.c file that kept track of how the cars and queues were being used. I also created a timestamps.c file that keeps track of how much time has elapsed. I think splitting my program up into multiple files helped make my project easier to understand and debug.

Performance: The project performed as expected and didn't have any memory leaks. The output printed fairly smoothly although it did buffer a bit sometimes.

Conclusion: This project helped me better understand threads in C. It also helped me to learn how to separate functionalities in a bigger project and this was very valuable.